

SALESTORM

Team : 5

Members : Akhilesh M P

Deepak R

Mohamed Fahim Jamal

High-Scale E-Commerce Flash Sale

System Design & Proof of Concept

Scalable • Consistent • Fault-Tolerant • Event-Driven

01. Executive Summary

SALESTORM is a high-scale e-commerce flash-sale architecture designed to handle extreme traffic bursts while maintaining strict inventory consistency and reliable payment and order processing.

The primary scenario is a flash sale in which thousands of customers compete simultaneously for a very small number of available units.

The system must solve four fundamental distributed-system problems:

1. Extreme request concurrency.
2. Limited inventory and overselling prevention.
3. Duplicate requests and payment retries.
4. Partial service failures during checkout and order creation.

SALESTORM addresses these challenges through:

- Edge traffic protection.
- Admission control and a waiting room.
- Atomic inventory reservation.
- Idempotency keys.
- Transactional outbox.
- Kafka-based asynchronous processing.
- Fault isolation and retries.
- Reservation expiry and compensation.
- Observability and invariant validation.

The central design principle is:

Scale aggressively around the consistency boundary, but never compromise correctness at the inventory and payment boundaries.

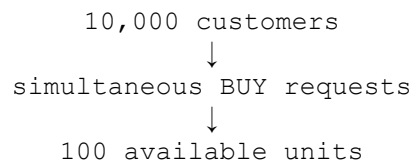
The uploaded simulator validates this design through a live request pipeline, inventory state visualization, chaos scenarios, event logs and invariant checks.

02. Problem Statement

Traditional e-commerce systems are designed for relatively predictable traffic.

A flash sale is fundamentally different.

A product with only **100 available units** may receive thousands of simultaneous purchase attempts.



Without careful concurrency control, the system can experience:

- Overselling.
- Duplicate reservations.
- Duplicate payment charges.
- Orders without successful payment.
- Lost payment/order state.
- Cascading failures.
- Database overload.

The system therefore requires a design where high throughput does not compromise correctness.

03. Requirements & Success Criteria

Functional Requirements

ID	Requirement
FR-01	Customer can browse products
FR-02	Customer can add products to cart
FR-03	Customer can participate in flash sale
FR-04	System can reserve inventory
FR-05	System can process payment
FR-06	System can create an order
FR-07	System can create shipment

ID	Requirement
FR-08	System can send notifications

Non-Functional Requirements

Category	Requirement
Scale	Support flash-sale traffic up to 500K RPS
Concurrency	Support 10K+ simultaneous buyers
Inventory	Zero overselling
Payment	No duplicate charges
Reliability	Recover from service failures
Availability	Graceful degradation
Security	TLS, authentication, authorization and WAF
Observability	Metrics, logs and distributed tracing
Recovery	Retry, replay and reconciliation

04. System Context

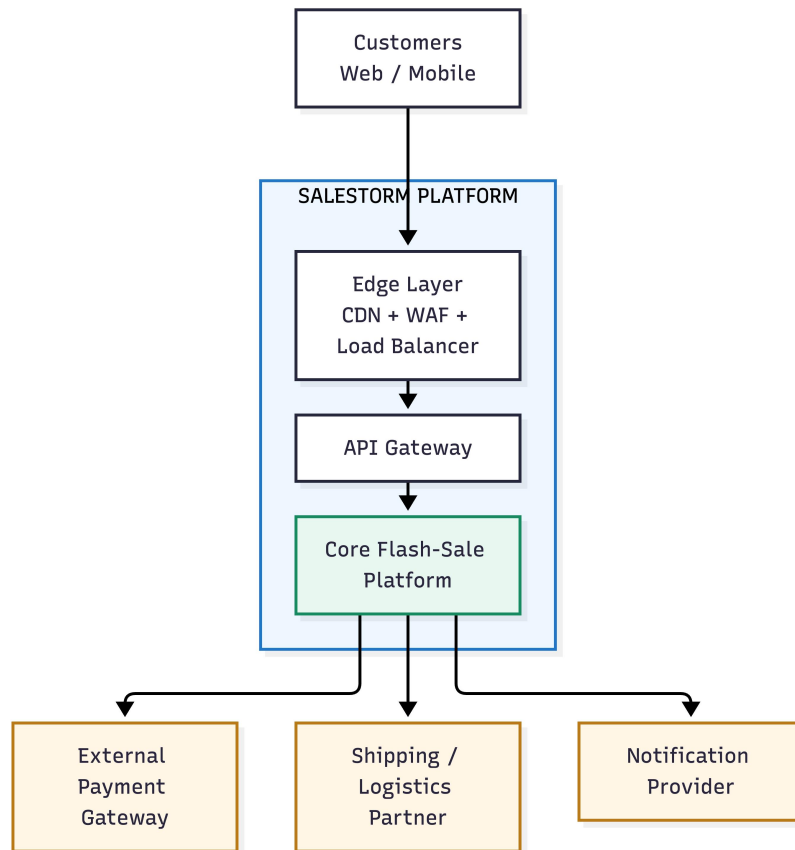


Figure 1

Description

SALESTORM sits between customers and external commerce providers.

Customers interact through web/mobile clients. The platform controls traffic admission, inventory, checkout, payment orchestration, order creation and fulfillment integration.

External payment, shipping and notification systems remain outside the SALESTORM trust boundary.

05. High-Level Architecture

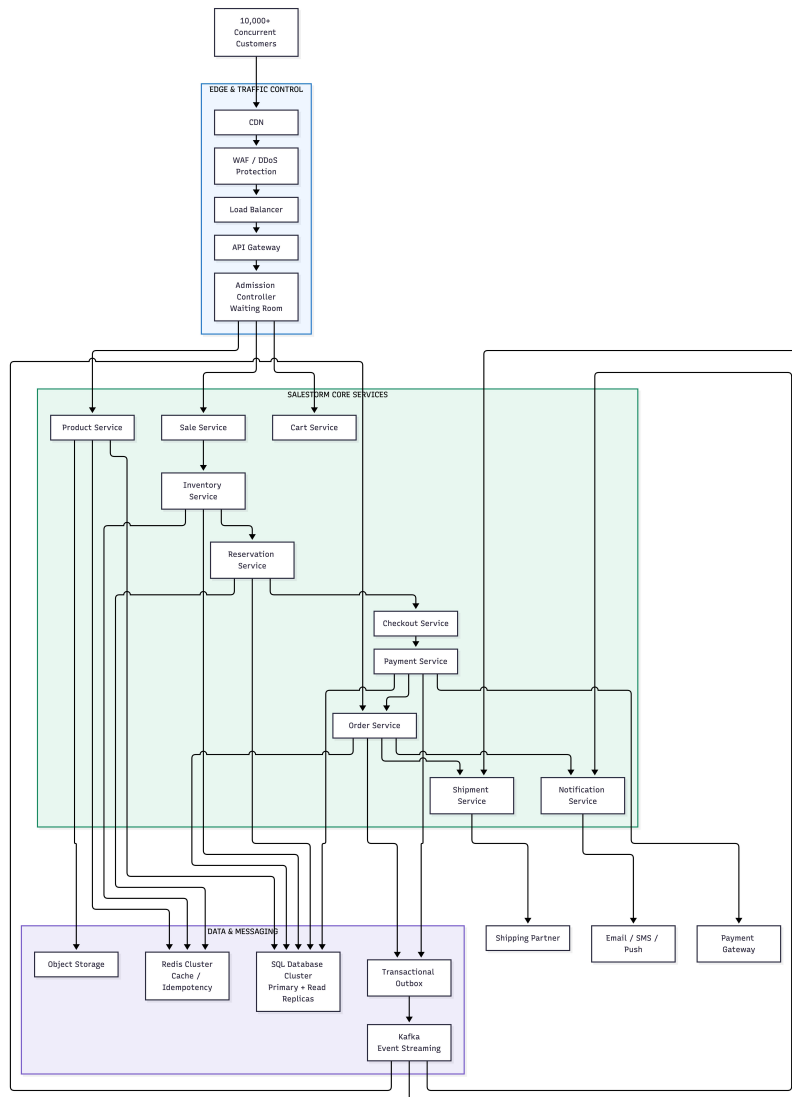


Figure 2

Architecture Layers

Edge Layer

Responsible for:

- CDN delivery.
- DDoS protection.
- WAF.
- Load balancing.
- API authentication.
- Rate limiting.

Business Layer

Contains:

- Product Service.
- Cart Service.
- Sale Service.
- Inventory Service.
- Reservation Service.
- Checkout Service.
- Payment Service.
- Order Service.
- Shipment Service.
- Notification Service.

Data & Messaging Layer

Contains:

- Redis.
- SQL database.
- Transactional outbox.
- Kafka.
- Object storage.

External Systems

- Payment gateway.
- Shipping provider.
- Notification provider.

06. Flash-Sale Admission Flow

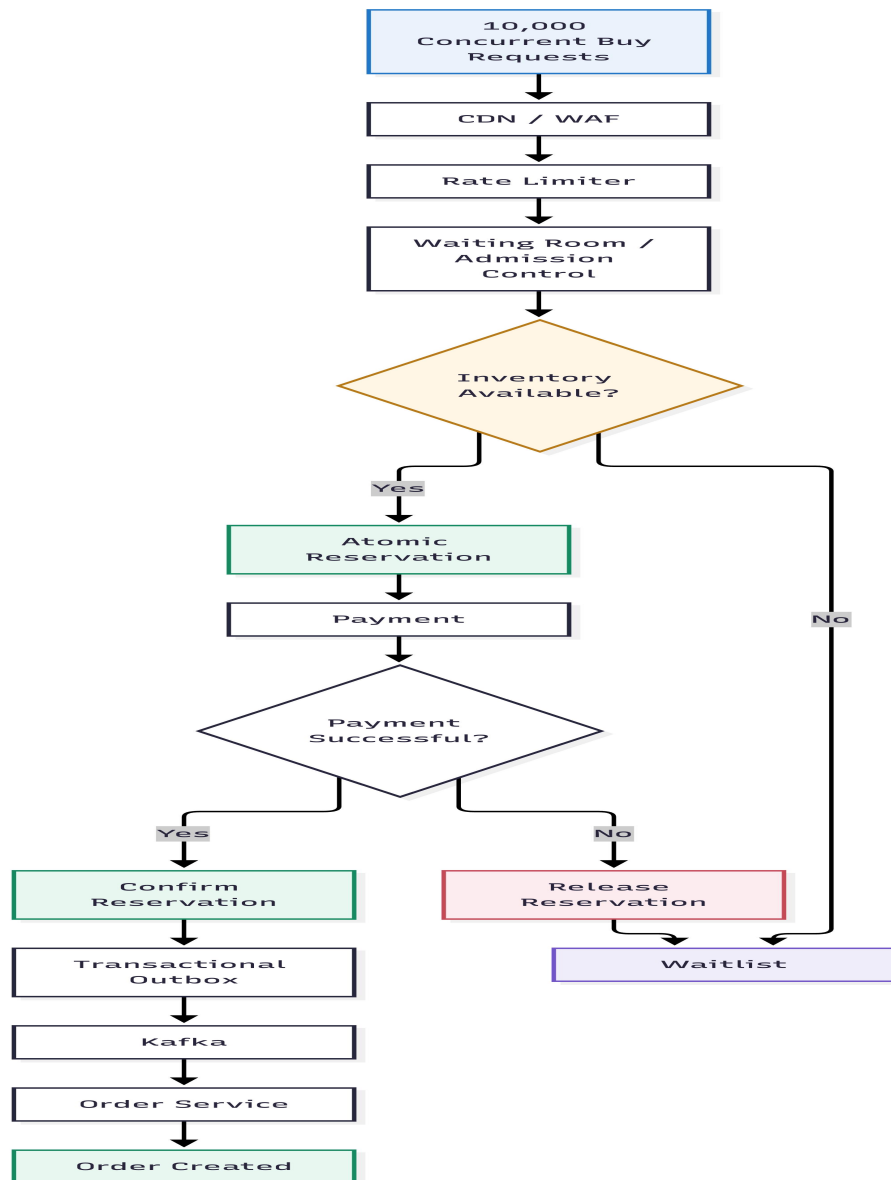
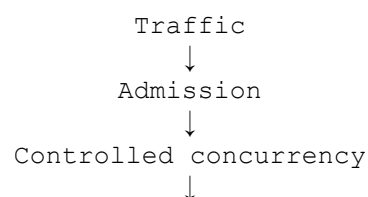


Figure 3

Why admission control?

Allowing all 10,000 users to directly hit the inventory database would create unnecessary contention.

Instead:



Inventory gate

The waiting room provides backpressure and protects the critical inventory path.

The simulator models an admission rate and queues requests before they reach the inventory gate.

07. Inventory Consistency

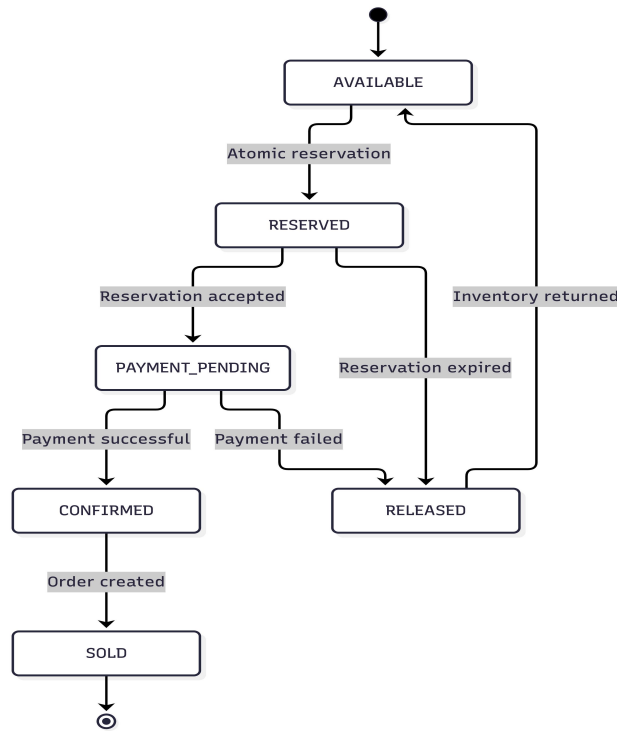


Figure 4

Inventory Model

```
available_quantity
reserved_quantity
sold_quantity
version
```

The authoritative inventory state resides in the database.

The reservation operation must be atomic.

Conceptually:

```
UPDATE inventory
SET available_quantity = available_quantity - 1,
    reserved_quantity = reserved_quantity + 1
WHERE product_id = ?
    AND available_quantity >= 1;
```

Result

Rows affected = 1
→ Reservation successful

Rows affected = 0
→ No inventory available
→ Waitlist / rejection

Inventory invariant

$\text{available} + \text{reserved} + \text{sold} = \text{total}$

and:

$\text{available} \geq 0$

The simulator continuously validates these conditions.

08. Idempotency

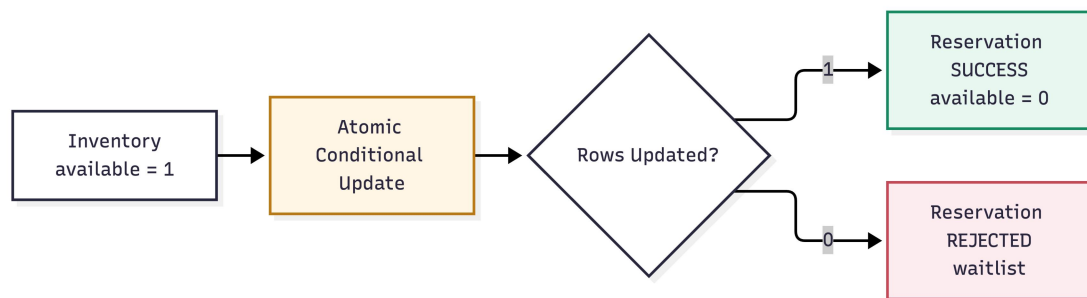


Figure 5

A client may send the same request multiple times because of:

- Double-clicks.
- Network retries.
- Browser retries.
- Mobile reconnects.
- Load balancer retries.

Each logical operation therefore receives an idempotency key.

Example:

Idempotency-Key: 7f91c...

If the same key is received again, the system returns the previously generated result rather than executing another transaction.

The simulator models duplicate requests and absorbs them before creating another reservation.

09. Payment & Order Architecture

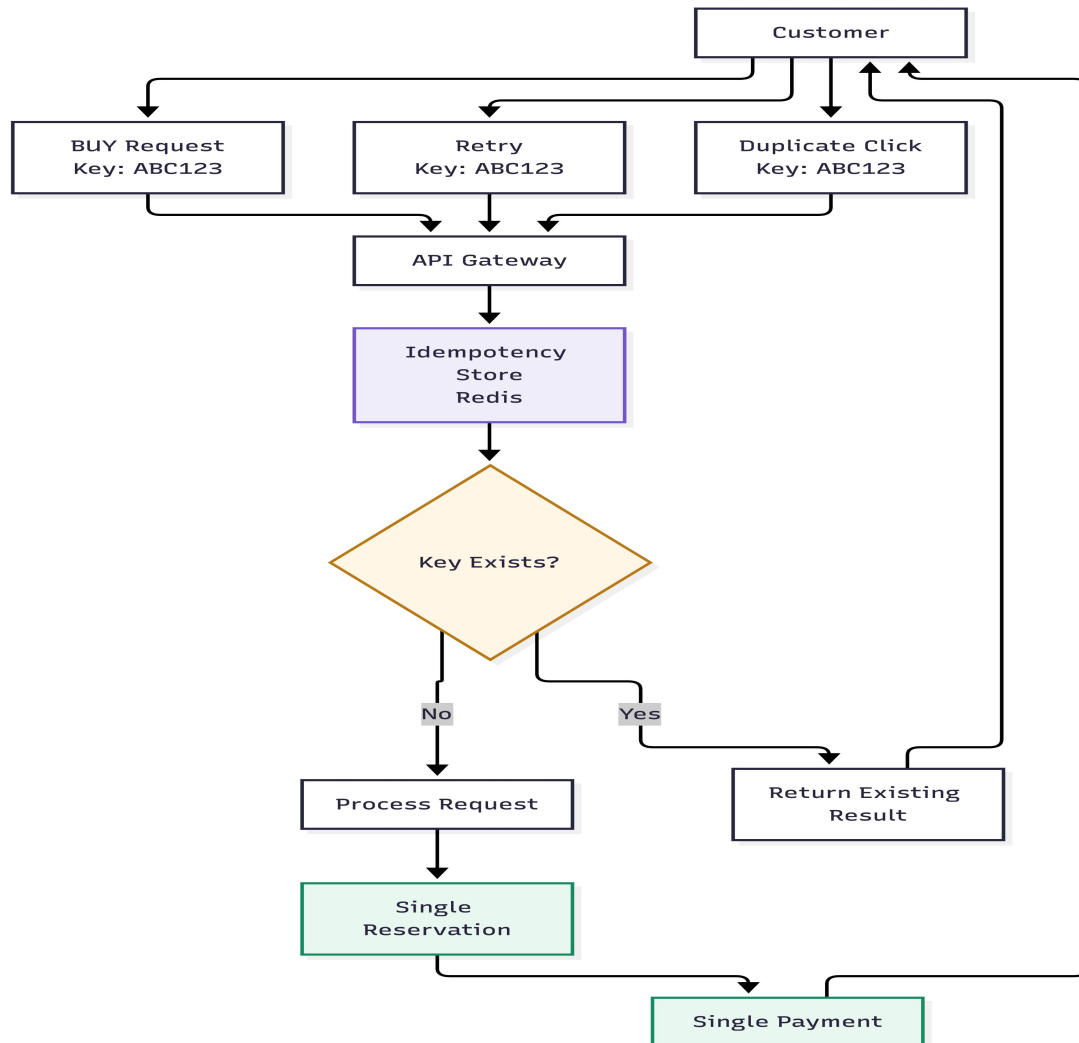
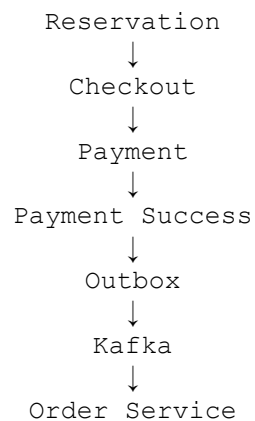


Figure 6

Payment flow



The important design decision is that payment success is persisted before the event is published.

This prevents the dangerous situation where:

```
Payment succeeded
↓
Order service failed
↓
Payment information lost
```

Instead:

```
Payment
↓
Durable state
↓
Outbox
↓
Kafka
↓
Order
```

The simulator represents successful payment as an event written to the outbox and subsequently consumed by the order path.

10. Failure & Recovery

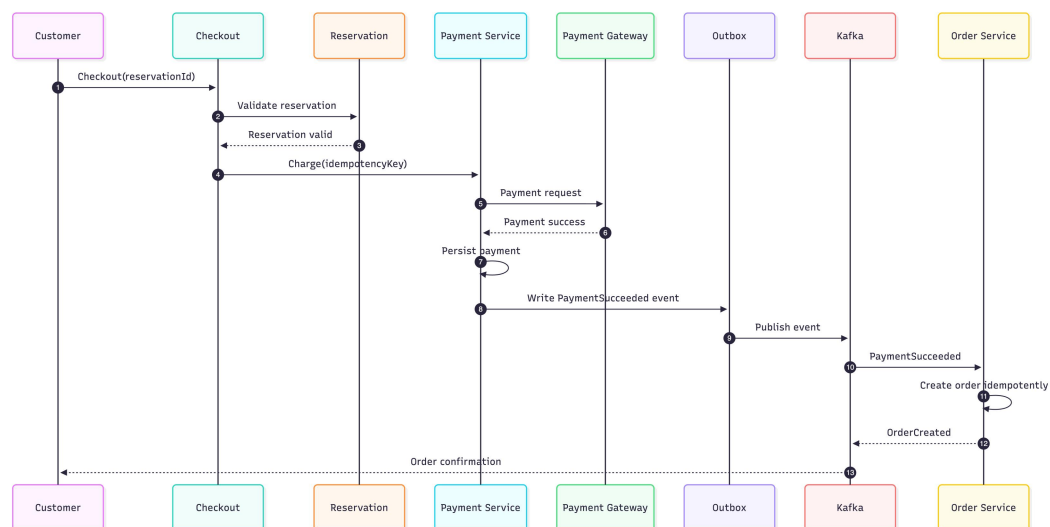


Figure 7

SALESTORM assumes failures will happen.

The architecture therefore uses:

Retry

For transient failures.

Exponential backoff

To prevent retry storms.

Circuit breaker

For repeatedly failing external services.

Outbox

To preserve events.

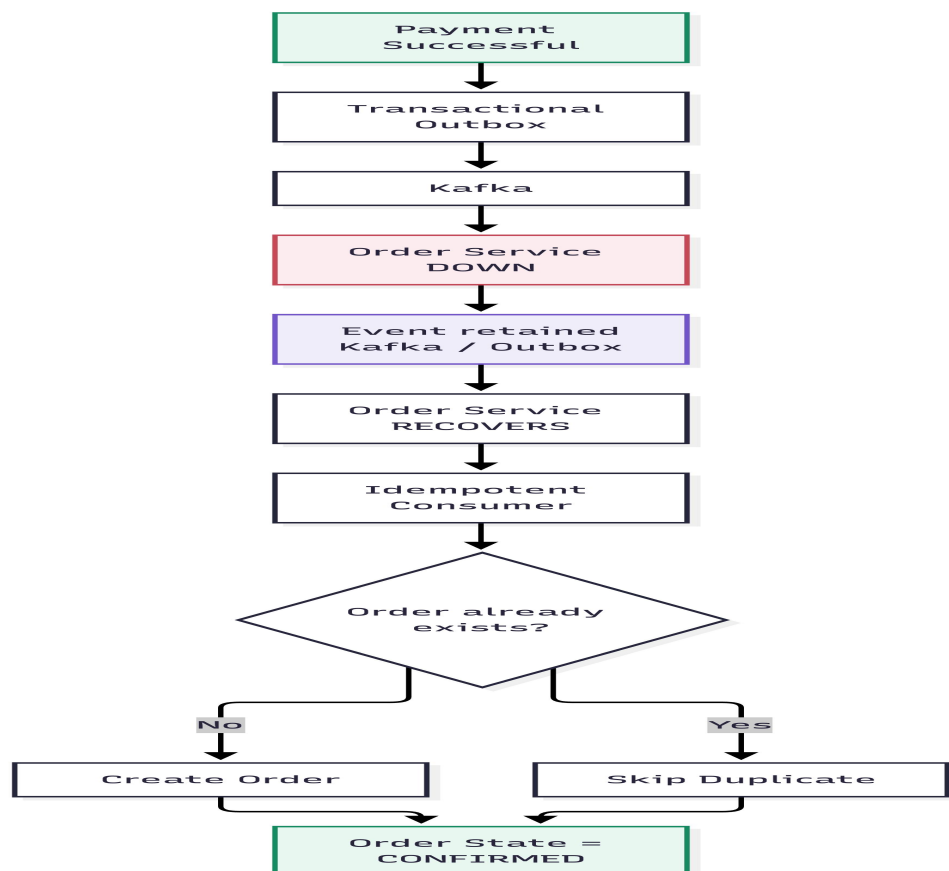
Kafka

To buffer asynchronous work.

Idempotent consumers

To safely replay events.

11. Data Architecture



Authoritative data

SQL is used for consistency-critical transactional state.

Inventory

Stores the authoritative stock state.

Reservation

Stores temporary ownership of inventory.

Payment

Stores payment transaction state.

Order

Stores the customer purchase state.

Shipment

Stores fulfillment information.

Redis is used for fast-access data such as:

- Caching.
- Idempotency records.
- Admission/rate-limiting state.

The database remains the authoritative source for inventory correctness.

11. API & Event Contracts

Core APIs

Create Reservation

```
POST /api/v1/reservations
```

Headers:

```
Authorization  
Idempotency-Key  
Request-ID  
Trace-ID
```

Request:

```
{  
  "productId": "P1001",  
  "quantity": 1  
}
```

Response:

```
{  
  "reservationId": "R12345",  
  "status": "RESERVED",  
  "expiresAt": "2026-10-05T10:30:00Z"  
}
```

Payment

```
POST /api/v1/payments  
{  
  "reservationId": "R12345",  
  "amount": 999  
}
```

Order

```
POST /api/v1/orders  
{  
  "paymentId": "PAY12345",  
  "reservationId": "R12345"  
}
```

Events

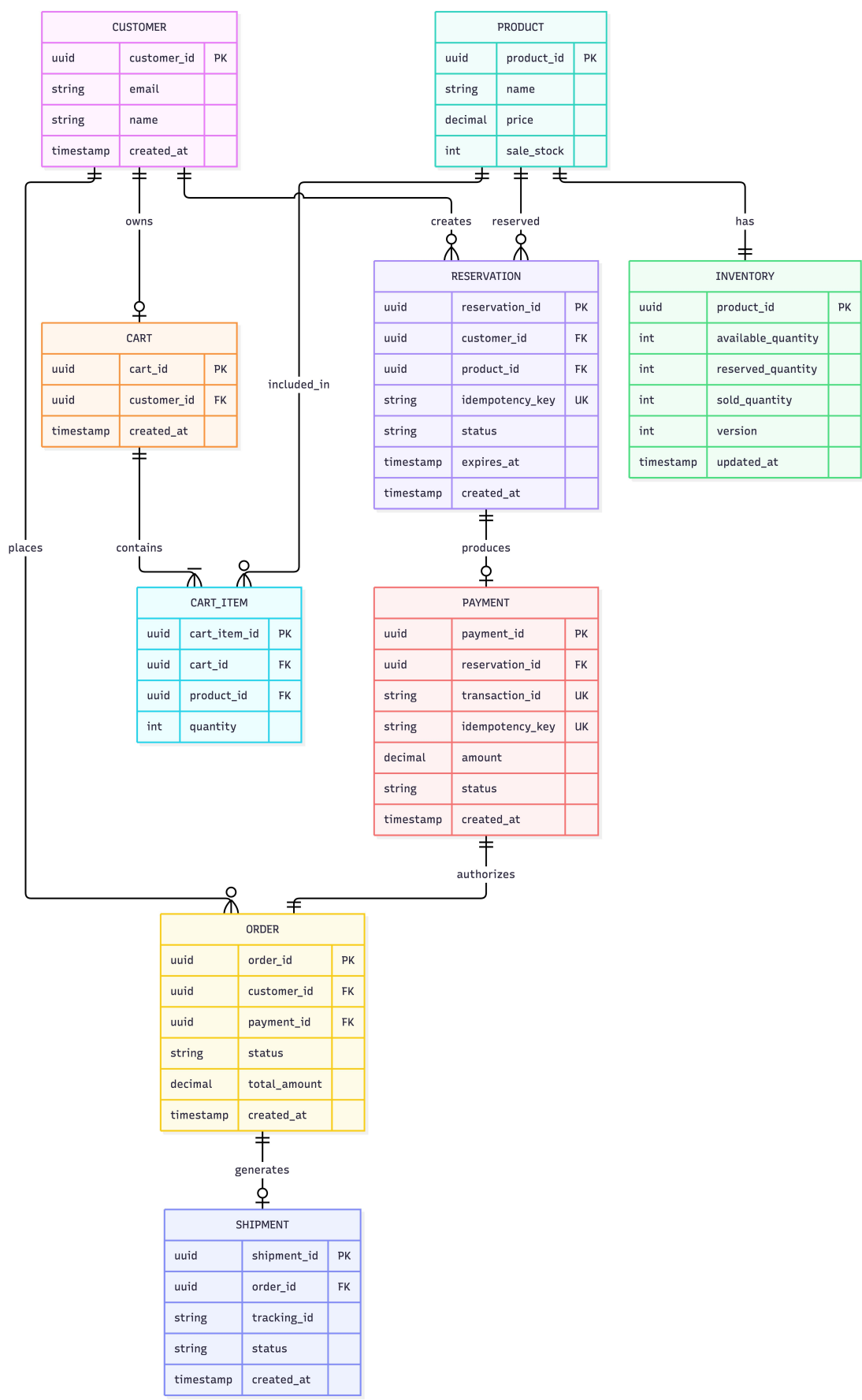
ReservationCreated
ReservationExpired
ReservationReleased

PaymentSucceeded
PaymentFailed

OrderCreated
OrderConfirmed
OrderCancelled

ShipmentCreated

13. Low-Level Design



The low-level design separates business logic from persistence and infrastructure.

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

The critical services use interfaces so implementations can be replaced without changing business logic.

Examples:

```
PaymentProvider
InventoryRepository
EventPublisher
IdempotencyStore
```

14. SOLID & Design Patterns

SOLID

Single Responsibility

Each service owns one major business capability.

Open/Closed

New payment providers can be added without modifying the core payment workflow.

Liskov Substitution

Payment provider implementations can be substituted behind a common interface.

Interface Segregation

Services depend only on the interfaces they require.

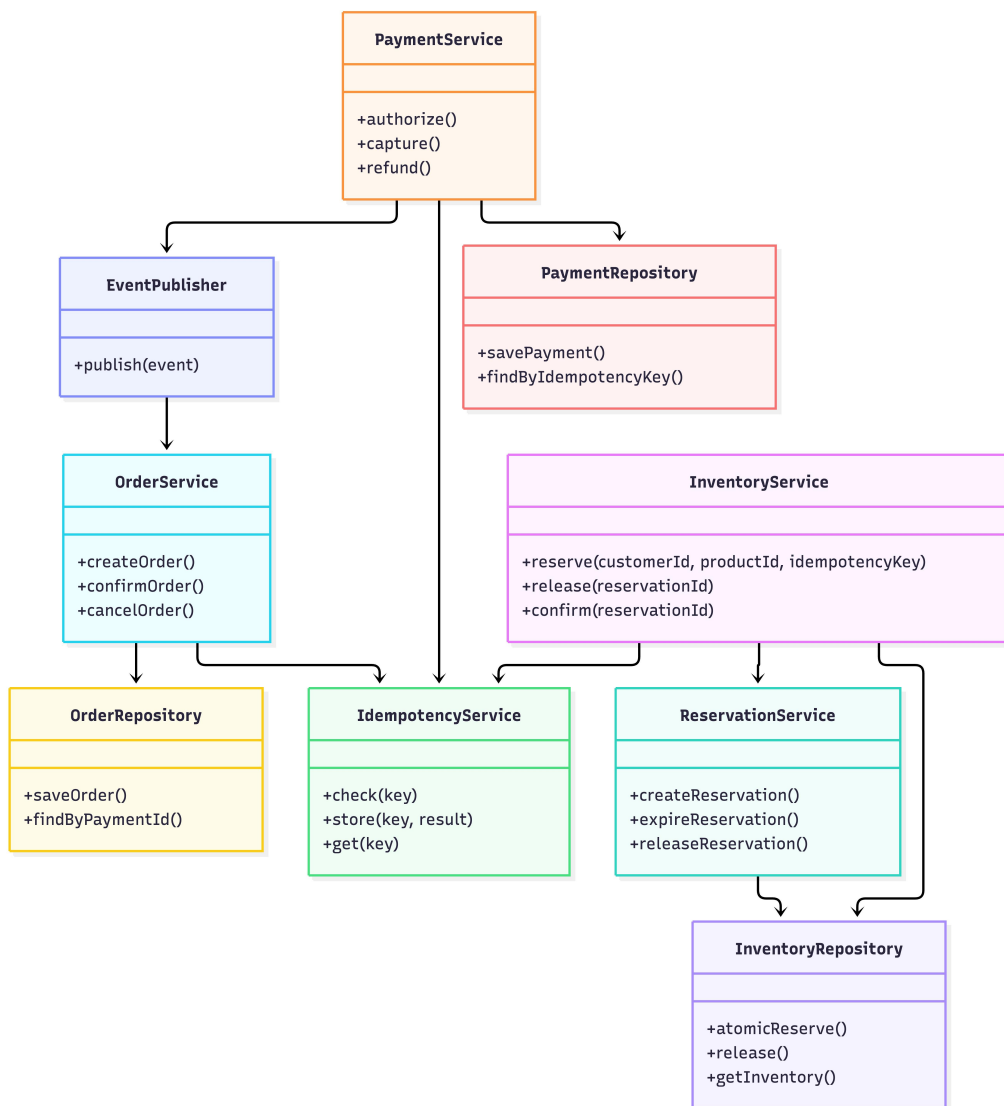
Dependency Inversion

Business services depend on abstractions rather than concrete infrastructure.

Design Patterns

Pattern	Application
Strategy	Payment provider selection
Adapter	External payment/shipping APIs
State	Reservation/order lifecycle
Repository	Database access
Factory	Provider creation
Circuit Breaker	External service protection
Observer/Event	Asynchronous processing
Facade	Checkout orchestration

15. Scalability Architecture



Horizontal scaling

Stateless API services can be replicated:

API 1
API 2
API 3
...
API N

Load balancing distributes traffic across instances.

Caching

Redis and CDN reduce repeated database reads.

Queue-based decoupling

Kafka absorbs asynchronous workloads and protects downstream services.

Database scaling

Use:

- Primary database for writes.
- Read replicas for read-heavy workloads.
- Partitioning/sharding when data volume requires it.
- Connection pooling.

Backpressure

The admission controller prevents downstream systems from being overwhelmed.

16. Security & Observability

Security

Network

- TLS/HTTPS.
- WAF.
- DDoS protection.

Application

- Authentication.
- Authorization.
- Input validation.
- Rate limiting.

Data

- Encryption at rest.
- Secrets management.
- Payment tokenization.
- Audit logs.

Observability

Every request should carry:

Request-ID
Trace-ID
Idempotency-Key

Metrics

Monitor:

Request rate
P95 latency
P99 latency
Queue depth
Reservation success rate
Payment success rate
Payment failure rate
Kafka lag
Database latency
Cache hit rate
Error rate

The simulator already exposes real-time counters, service health, event logs and trace information, making it useful as the POC validation layer.

SALESTORM_Live_Simulation_Conso...

17. POC Implementation

SALESTORM Live Simulator

Insert your **actual professional HTML simulator screenshot** here.

Do **not** generate an AI diagram for this section.

Add these callouts:

- Request Pipeline
- Waiting Room
- Atomic Inventory Gate
- Payment / Outbox
- Inventory Ledger
- Live Counters
- Invariant Checks
- Event Log
- Chaos Controls

The uploaded implementation includes the request pipeline, chaos controls, inventory state, system health, correctness checks and simulation verdict.
SALESTORM_Live_Simulation_Conso...

18. Simulation Results

Your simulator should produce a results table.

Use:

Metric	Result
Total Customers	10,000
Inventory	100
Successful Sales	100
Oversells	0
Duplicate Charges	0
Orders Without Payment	0
Duplicate Requests Absorbed	N
Reservations Expired	N
Waitlist Promotions	N
Order Retries	N

Then:

✓ ALL INVARIANTS HOLD

The simulator generates a final verdict only after requests, payments and orders settle, and explicitly reports sold units, oversells, duplicate charges, orders, duplicate

requests, expirations, waitlist promotions and retries.
SALESTORM_Live_Simulation_Conso...

Important: Replace N with the actual values from your final run. Don't fabricate these numbers.

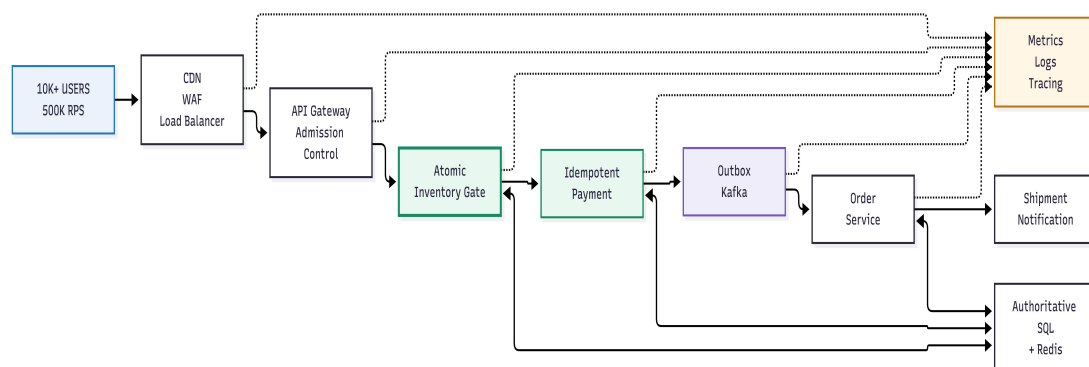
19. Architecture Decisions & Trade-offs

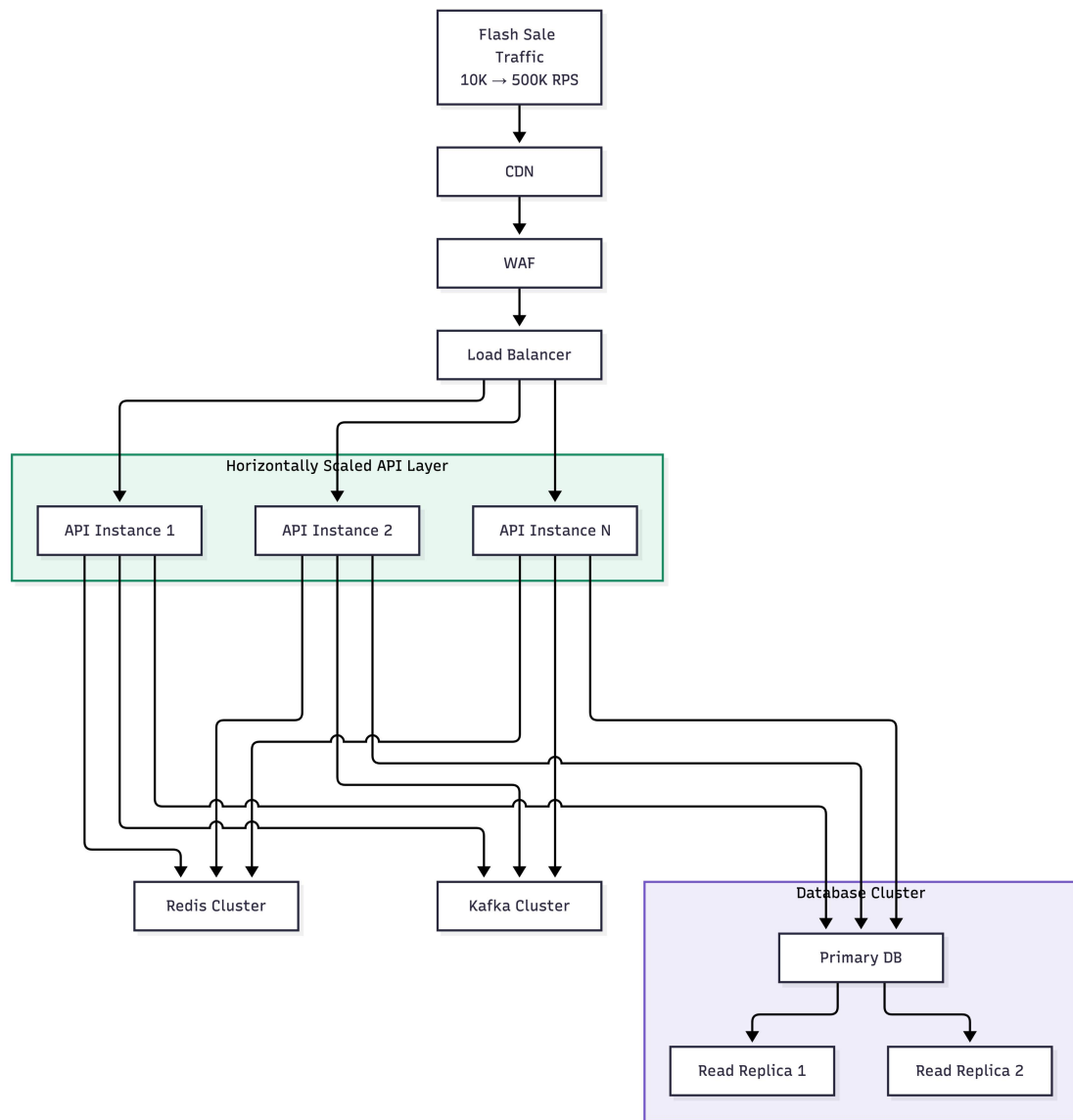
Decision	Alternative	Reason
SQL for inventory	NoSQL	Strong consistency
Atomic DB update	Distributed locking	Simpler correctness boundary
Redis	Database-only	Reduce hot-path load
Kafka	Fully synchronous calls	Decoupling and buffering
Outbox	Direct publish	Prevent lost events
Idempotency	Blind retries	Prevent duplicate operations
Waiting room	Direct admission	Protect critical services
Horizontal scaling	Vertical scaling	Better elasticity

Core trade-off

SALESTORM does not attempt to maximize availability at the expense of inventory correctness. The inventory boundary is deliberately strict, while non-critical processing is made asynchronous and horizontally scalable.

20. Final Architecture Summary





Final Guarantees

- ✓ ZERO OVERSELLING
- ✓ ZERO DUPLICATE PAYMENT CHARGES
- ✓ NO ORDER WITHOUT PAYMENT
- ✓ IDEMPOTENT REQUEST PROCESSING
- ✓ RESERVATION EXPIRY
- ✓ EVENTUAL ORDER RECOVERY
- ✓ HORIZONTAL SCALABILITY
- ✓ FAILURE ISOLATION

✓ OBSERVABLE SYSTEM STATE

Final statement

SALESTORM converts an uncontrolled flash-sale burst into a controlled, consistent and recoverable distributed workflow.

10,000+ concurrent customers can compete for 100 units while the system preserves inventory correctness, prevents duplicate payment, survives downstream failures and eventually converges to a consistent order state.